Summarising Football Match Commentaries: A Comparative Study of Prompting and Fine-tuning on Long-form Sports Audio Transcripts

Marco Panciera Christian Dalfarra
Applied Natural Language Processing
University of Trento

May 2026

Abstract

We study automatic summarisation of long football-match commentaries, where the input is a Whisper transcription of ∼2.5 hours of live BBC radio audio (mean ∼25 000 words) and the target is a 150–500 word match report. We construct a custom dataset of 99 matches spanning 1983–2026 and 12 competitions, with summaries generated by prompting search-enabled GPT-4 on a per-match basis — grounded in web-retrieved match reports rather than the transcript — and used as references during both training and evaluation. We compare two orthogonal design axes: *input handling* (chunk-and-aggregate vs. long-context with LED-16384) and *learning paradigm* (zero/few/CoT prompting vs. supervised fine-tuning) across three architectures (FLAN-T5-Large, BART-Large-CNN, LED-base). Fine-tuning BART as the *merge step* of a chunk-aggregate pipeline yields ROUGE-L = 0.248, a +55% relative improvement over the strongest prompting baseline and +69% over the same pre-trained BART under identical inference conditions. We further document a costly empty-reference dataset bug that masked the true performance ceiling by a factor of ∼1.8× for two days, and report extensive qualitative evidence that all fine-tuned models, while learning the surface form of a match report, hallucinate scorers, scores, and stadiums — a failure mode that ROUGE rewards rather than penalises.

1 Introduction

Automatic summarisation of long-form spoken-language transcripts is a stress test for modern sequence-to-sequence models: inputs routinely exceed the context window of a standard transformer, outputs require selective abstraction over hours of fragmented commentary, and references for training data are scarce. Football match commentaries are an especially appealing test bed because (i) the input is structurally noisy (live audio, overlapping speakers, prosodic emphasis on irrelevant detail), yet (ii) the desired output — a match report — has a stable, learnable surface format.

This project asks two narrowly-scoped, empirically answerable questions:

1. Given a ∼25 000-word transcript and an output budget of ∼300 words, is it better to *chunk and aggregate* or to use a *long-context* encoder?
2. For each input strategy, does *supervised fine-tuning* on a small in-domain dataset outperform *prompting* of a general-purpose model?

We instantiate the cross product on three architectures (FLAN-T5-Large, BART-Large-CNN, LED-base-16384) and three prompting strategies (zero-shot, few-shot, chain-of-thought (Wei

et al., 2022)), giving eleven model configurations (nine prompting, two fine-tuned) evaluated on a held-out 9-match test set.

Contributions.

- A custom 99-match dataset (BBC radio audio → Whisper transcript → GPT-4 reference summary), released under permissive licensing.
- A *chunk-merger* fine-tuning recipe for BART in which the training input is the concatenation of pre-trained per-chunk summaries, eliminating the train/inference distribution mismatch of naive truncated-input training and improving ROUGE-L by +69% over the same pre-trained model.
- A systematic comparison of prompting vs. fine-tuning across two input pipelines, with qualitative analysis exposing hallucination failures that aggregate metrics obscure.
- A documented engineering history of five distinct silent failures encountered during LED fine-tuning, of independent practical value for small-scale long-document fine-tuning.

2 Related Work

Long-document summarisation. Standard transformer summarisers (BART (Lewis et al., 2020), PEGASUS, T5) cap input at 1024 tokens, far below the ∼30 000-token regime of full-match transcripts. Two families of solutions have emerged: (i) extend the attention mechanism to support longer contexts (Longformer / LED (Beltagy et al., 2020)); (ii) chunk the document, summarise each piece, and recombine (See et al., 2017). We compare both directly on the same data.

Prompting vs. fine-tuning. Instruction-tuned models (FLAN-T5 (Chung et al., 2022), GPT-style models (Brown et al., 2020)) can perform many summarisation tasks zero-shot. Fine-tuning on small in-domain data, however, often outperforms prompting when the target distribution differs systematically from web-scale pre-training data. Sports commentary is one such case: pre-training corpora contain match reports as *outputs* but rarely raw radio transcripts as inputs.

Faithfulness. Maynez et al. (2020) document that abstractive summarisers routinely introduce content not entailed by the source. ROUGE (Lin, 2004) cannot detect this; BERTScore (Zhang et al., 2020) captures semantic but not factual correctness. We provide qualitative evidence of hallucination in our trained models and discuss this limitation in Section 9.

3 Dataset Construction

3.1 Source selection and audio collection

We selected 100 football matches[1] from the publicly-available BBC Radio 5 Live archive on YouTube, spanning 43 years (1983–2026) and 12 competitions: World Cup (32), UEFA Champions League (17), Premier League (15), Euros (14), UCL final (8), WC qualifiers (5), FA Cup (2), Nations League (2), UEFA League Final, Euro final, WC final, UECL final (1 each). The mix is England-heavy — most fixtures involve a home-nations team, reflecting the source — with a deliberate inclusion of neutral fixtures (Spain–Germany, Argentina–Poland) to

[1] One audio file failed extraction and was dropped, leaving 99 usable matches.

test out-of-distribution generalisation. Matches were downloaded as MP3 via `yt-dlp` (script `scripts/youtube_download.py`), totalling ∼19 GB of audio.

3.2 Speech-to-text transcription

Each MP3 was processed by OpenAI Whisper `small` (Radford et al., 2023) (script `scripts/transcribe_batch.py`). For every Whisper segment, we additionally extracted per-segment prosodic features — mean pitch (Hz) and RMS energy — using `librosa`. Each match produces three artefacts:

- `{id}_transcript.txt` — the full, unsegmented commentary (mean 130 KB, ∼25 000 words).
- `{id}_segments.txt` — the same text split into time-aligned Whisper utterances with pitch and energy annotations.
- `{id}.json` — structured token-level alignment.

A representative segment line:

```
[0.00 - 6.80] P:1767.91 E:0.031 | Wenger's Arsenal look like this, Seaman in
goal, a back four of Dixon, Keown, Adams and Cole.
```

Whisper transcription introduces systematic errors on player names ("Bergkamp" → "Burgkamp", "Wenger's" → "Vengas") that propagate through every downstream pipeline and bound the achievable surface-token overlap from above.

3.3 Reference summary generation

The dataset has no naturally-paired ground-truth summaries for radio commentary. We therefore generated reference summaries by prompting GPT-4 (OpenAI, 2023) on a per-match basis via the ChatGPT web interface with search enabled: for each match the model was given the match identifier (year, competition, teams) and instructed to retrieve contemporary accounts of the fixture before writing a structured summary. We manually checked the citations returned for every match and confirmed that each report drew on the BBC website among its sources. Grounding generation in retrieved sources anchors the factual content (scores, scorers, key events) in published accounts rather than the model's parametric memory. The output template is fixed:

> *Line 1: {TeamA} {scoreA}–{scoreB} {TeamB}*
> *Line 2: {Competition}, {Stadium}, {City} – {Date}*
> *Lines 3+: 2–4 paragraph report covering result, key events, and a brief tactical note.*
> *Final block: bullet list of key events (e.g. "27' – Gordon Smith goal").*

Mean reference length is ∼1.2 KB (∼300 words). One representative reference:

> *Brighton & Hove Albion 2–2 Manchester United (after extra time)*
> *1983 FA Cup Final, Wembley Stadium, London – 21 May 1983*
> *The 1983 FA Cup Final between Brighton & Hove Albion and Manchester United ended in a dramatic 2–2 draw, forcing a replay... Brighton took the lead in the 27th minute when Gordon Smith finished clinically...*

Methodological note. Using a large language model to generate references introduces an evaluation bias even with retrieval: the references reflect web-sourced match reports as filtered and rewritten by GPT-4 at generation time, and they remain *ungrounded in the transcript* our models actually read — events a radio commentator dwells on may be absent from a written

report and vice versa. Retrieval also weakens reproducibility: search results change over time, so regenerating the references would not yield byte-identical targets. We discuss this candidly in Section 9. The alternative — human-written summaries for 99 matches — was outside the scope of a course project. References were spot-checked manually and accepted as a working approximation.

3.4 Splits and statistics

A stratified train/val/test split of 80/10/9 was generated once (`scripts/generate_splits.py`) and committed to the repository to ensure reproducibility. Test matches deliberately span eras and competitions (1983 FA Cup, 2001 FA Cup, 2019 UCL Final, 2021/2022/2024 internationals, 2026 UCL group stage). Dataset statistics are summarised in Table 1.

Property	**Value**
Matches	99
Year coverage	1983–2026 (43 years)
Competitions	12
Train / val / test	80 / 10 / 9
Mean transcript length	130 KB (∼25 000 words)
Mean segments per match	∼700
Mean reference length	1.2 KB (∼300 words)
Total dataset size (excl. raw audio)	91 MB
Raw audio (gitignored)	∼19 GB

Table 1: Dataset statistics for the in-project `football_commentary_dataset 1.0`.

4 Methodology

We compare two input-handling pipelines and two learning paradigms, giving four cells. Three architectures are evaluated where appropriate, with each prompting condition further crossed with three prompt styles (zero, few, CoT).

4.1 Two input pipelines

Chunk-and-aggregate. The segmented transcript is split into contiguous chunks of $\leq M$ model tokens with a one-segment overlap to preserve cross-chunk context. A generator function $g(\text{chunk}) \rightarrow$ partial summary is applied to each chunk, and a merger function $m(\text{partial}_1, \ldots, \text{partial}_n) \rightarrow$ final summary produces the final output. We use:

- $M = 450$ for FLAN-T5 (512-token limit minus prompt overhead),
- $M = 900$ for BART (1024 minus output budget).

After merging, a bag-of-words cosine deduplication pass removes near-duplicate sentences across the concatenated partial summaries.

Long-context. The full transcript is passed to LED-base-16384 (Beltagy et al., 2020) in a single forward pass. LED's sliding-window local attention plus full global attention on a designated BOS token make this tractable on a single T4 GPU. No chunking, no merge step.

4.2 Prompting setup

We test three prompt styles, fixed per model family for fair comparison:

- **Zero-shot.** An instruction ("Summarise the following football commentary into a concise match report…") followed by the chunk or transcript.
- **Few-shot.** The zero-shot prompt prefixed with two in-context (commentary → summary) examples drawn from the training set.
- **Chain-of-thought.** A structured instruction asking the model to first enumerate goals, cards, and substitutions, then write the prose summary.

Decoding parameters are held constant across conditions: beam size 4, length penalty 1.0, no-repeat n-gram size 3, max output 256 tokens for chunk-level generation and 512 for the merge step.

4.3 Fine-tuning setup

BART chunk-merger fine-tuning. A naive fine-tuning recipe — train BART on (transcript[:1024], gold) pairs — exposes the model to only the first ∼5 minutes of a 2.5-hour match. To match the train and inference distributions, we instead train BART as the *merge step* of the chunk-aggregate pipeline:

1. For each training match, run pre-trained BART-Large-CNN on every chunk to obtain ∼30 mini-summaries of ∼32 tokens each.
2. Concatenate the chunk summaries into a single ∼960-token intermediate.
3. Train BART to map (intermediate → gold reference) with a two-phase schedule: encoder frozen for 3 epochs (decoder adapts to the new target distribution), then fully unfrozen for the remainder. Early stopping on validation ROUGE-L with patience 3.

The chunk-summary generation is cached on disk because it is the dominant one-time cost. Hyperparameters: learning rate 3×10^{-5}, effective batch 8 (batch 1 with gradient accumulation), `fp16`, max input 1024 tokens, max target 256 tokens.

LED long-context fine-tuning. LED-base is fine-tuned directly on (full transcript → gold) pairs, with a custom random-window sampler that, each epoch, samples a contiguous 8192-token window starting at a uniformly random offset within transcripts longer than the window. This effectively augments the training set by exposing the model to different portions of each match. Global attention is forced on the BOS token; gradient checkpointing is enabled to fit a single sequence on a T4. Hyperparameters: learning rate 5×10^{-5}, batch 1 + grad-accum 8, `fp16`, max input 8192 tokens, max target 512 tokens, evaluation generation floor `min_length=80` to expose collapse-to-empty failures (see Section 8).

4.4 Evaluation

We report ROUGE-1, ROUGE-2, and ROUGE-L F-measures (stemmed) averaged over the 9 test matches, using the `rouge_score` package (Lin, 2004). BERTScore (Zhang et al., 2020) was attempted but Kaggle's transformers/bert_score build raises an `OverflowError` on long inputs; we report ROUGE-only and discuss the limitation explicitly.

5 Experimental Setup

Compute. All experiments ran on free-tier Kaggle Notebooks with a single NVIDIA Tesla T4 (15.6 GB VRAM). The 12-hour-per-session and ∼30-GPU-hour-per-week limits dictated the experimental cadence: prompting baselines and BART fine-tuning together fit in one ∼2.5-hour session; LED fine-tuning is a separate ∼50-minute session.

Reproducibility. Random seeds are fixed to 42; the train/val/test split is committed; intermediate predictions are saved to disk so the evaluation step can re-run without regenerating outputs. The full data, code, and trained models are versioned at branch `setup/local-run` of the project repository.

Models. Table 2 summarises the architectures used.

Model	Role	Max input	Params
FLAN-T5-Large (Chung et al., 2022)	Prompting (chunk)	512	780M
BART-Large-CNN (Lewis et al., 2020)	Prompting + fine-tune (chunk)	1024	406M
LED-base-16384 (Beltagy et al., 2020)	Prompting + fine-tune (long)	16 384	162M

Table 2: Models evaluated, by role and context-window size.

6 Results

6.1 Main leaderboard

Table 3 reports ROUGE F-measures on the 9-match test set after the data quality fix described in Section 8. Fine-tuned BART (chunk-merger, two-phase) is the best overall by a wide margin.

Rank	Condition	ROUGE-1	ROUGE-2	ROUGE-L
1	**finetuned_bart** (chunk-merger, 2-phase)	0.4205	0.1330	**0.2476**
2	finetuned_led (long-context)†	0.4170	0.1250	0.2190
3	led_long_few	0.2712	0.0531	0.1599
3	led_long_zero	0.2712	0.0531	0.1599
5	led_long_cot	0.2773	0.0559	0.1548
6	bart_chunk_zero	0.2249	0.0516	0.1467
7	bart_chunk_cot	0.1989	0.0408	0.1230
8	bart_chunk_few	0.1765	0.0188	0.1030
9	flan_chunk_zero	0.1164	0.0212	0.0700
10	flan_chunk_few	0.0770	0.0086	0.0551
11	flan_chunk_cot	0.0733	0.0148	0.0481

Table 3: Main results: ROUGE F-measures (stemmed). Bold indicates the best condition. Every row except `finetuned_led` is a clean 9-match evaluation from the full re-run (run 4). † `finetuned_led` was trained in a separate, expensive Kaggle session (run 3) on the *pre-fix* dataset and was not re-trained after the empty-reference fix; its value is the valid-only re-evaluation over the 5 test matches with non-empty references, which removes the empty-reference penalty but is *not* strictly comparable to the 9-match clean protocol used for every other row. It is shown greyed to signal this. See Section 8.

6.2 Three central observations

(1) Fine-tuning beats the strongest prompting baseline by 55%. Fine-tuned BART (0.248) versus LED few-shot, the best prompting condition (0.160), is a +55% relative gap. The same-model comparison — pre-trained BART under the identical chunk-aggregate pipeline (0.147) — gives an even cleaner +69%, isolating the contribution of fine-tuning from the choice of model and pipeline. Both fine-tuned architectures (BART, LED) reach the top of the leaderboard, occupying positions 1 and 2.

(2) Long-context dominates among prompting baselines, but loses among fine-tuned models. Across the nine prompting cells, LED long-context averages ROUGE-L 0.158 versus 0.124 for BART chunk-aggregate and 0.058 for FLAN chunk-aggregate. The 16 384-token window preserves cross-segment information that chunking discards. Among fine-tuned models the ranking inverts: BART chunk-merger reaches 0.248 on the clean 9-match test set, while fine-tuned LED reaches 0.219 on its valid-only re-evaluation (5 matches; run 3 was not re-trained after the empty-reference fix, so the two numbers are close but not strictly comparable — see Section 8). Even granting LED the benefit of the comparability gap, BART is at least as strong. The likely explanation is sample efficiency: with only 80 training pairs, BART operates on a ∼960-token pre-distilled intermediate whose features are already informative, while LED must learn to attend across raw 8192-token windows from scratch.

(3) Zero-shot beats few-shot and CoT across every model family. This is a robust secondary finding (Table 4). In the chunk-aggregate pipeline, few-shot examples consume 200–400 tokens of the input budget, leaving less room for the actual transcript; CoT prompts further push generation toward verbose intermediate reasoning that the merge step must subsequently compress. In LED long-context, all three prompt styles converge to similar (poor) outputs because LED-base is not pre-trained on summarisation — the prompt simply does not reach the language model in a useful way.

Family	Zero	Few	CoT
BART (chunk)	**0.147**	0.103	0.123
FLAN (chunk)	**0.070**	0.055	0.048
LED (long)	**0.160**	0.160	0.155

Table 4: ROUGE-L by prompt style and model family. Zero-shot is best in every family.

7 Qualitative Analysis

Aggregate ROUGE numbers mask large differences in output *quality*. We inspected predictions for all 9 test matches; Table 5 shows representative outputs for the 2001 FA Cup Final (Arsenal–Liverpool, true score 1–2 with Owen scoring twice late on against Ljungberg's opener).

What we observe.

- **FLAN-T5** produces short, stylistically wrong (radio-broadcast snippet) and factually wrong content. With few-shot prompting, it copies the example target verbatim rather than summarising, a classic template-completion failure.
- **LED prompting** generates fragments of mid-commentary text that mimic the input style rather than summarising it. The model is not pre-trained for summarisation and out-of-the-box behaves as a continuation language model.

Model	Output (first 250 chars)
`flan_chunk_zero`	"Mark Warburton and Jamie Vardy contributed to BBC Radio 5 live coverage of Arsenal drew 1–1 at Anfield on Saturday."
`led_long_zero`	"... ope, and now to his right, the ball is going to the right side of the penalty area, it's going to be difficult to get the ball back to the left side..."
`finetuned_bart`	"Liverpool 1–1 Arsenal (Liverpool win 2–3 on penalties) 20 May 2014 – Emirates Stadium, London. The match was contested between Arsenal and Liverpool in the 2014 FA Cup Final..."
`finetuned_led`	"Liverpool 1–0 Arsenal, Premier League, 20 May 2001 – Anfield, Anfield, Emirates Stadium, Anfield – Anfield – 19 May 2001. Liverpool dominated the first half..."

Table 5: Representative outputs on the 2001 FA Cup Final test instance.

- **Fine-tuned BART** produces structured, fluent match-report prose with team names, score format, date format — and *invents* the year (2014, actually 2001), the stadium (Emirates, actually Millennium), and the result (penalty shootout, actually 1–2 in normal time).
- **Fine-tuned LED** similarly produces fluent match-report prose. It *recovers* the correct year (2001) but repeats "Anfield" four times, reports the wrong score (1–0 actually 1–2), and switches competitions (Premier League actually FA Cup).

Both fine-tuned models have learned the *format* of a match report. Neither has learned *factual fidelity*. ROUGE rewards them anyway because the surface n-grams that match the gold reference (team names, "in the X-th minute", stadium types, date formats) are exactly the part the models do reproduce. The hallucinated facts contribute to length but not to overlap, so they impose no metric penalty.

8 Engineering History

The path to the results in Section 6 included several costly silent failures. We document them because each has independent reproducibility value and because the engineering choices behind the final numbers are not visible in the metrics.

8.1 LED fine-tuning: five silent failures

The first three attempts to fine-tune LED collapsed: evaluation ROUGE-L dropped to zero by epoch 3 even as evaluation *loss* continued to decrease.

- **Bug 1 – defeated random-window sampler.** `scripts/run_finetuning.py` pre-truncated each transcript to 8192 tokens *before* handing it to the dataset's random-window sampler. The sampler then ran on inputs of length 8192 with window size 8192, so the random offset was always zero. The model saw the first ∼8 minutes of every match, every epoch.
- **Bug 2 – collapse-to-EOS under cross-entropy.** With the truncation bug active, the model found that emitting `<s></s>` immediately was an entropy-minimal local optimum. Greedy decoding then produced empty strings, giving ROUGE-L = 0 while `eval_loss` continued to improve. Adding `generation_config.min_length=80` forced the decoder to produce content and exposed the collapse.
- **Bug 3 – `OverflowError` in metric computation.** The trainer's prediction tensors occasionally contained −100 sentinels in *predictions* (in addition to labels). `tokenizer.batch_decode` (fast tokenizer, Rust backend) interprets negative indices as `u32` and overflows. Fix:

`np.where(preds<0, pad_id, preds)` applied to predictions, mirroring the existing fix on labels.

- **Bug 4 – missing checkpoint on crash.** The `OverflowError` occurred mid-evaluation, before `trainer.save_model()` ran. Inference then 404'd looking for `model.safetensors`. Fix: `run_inference_finetuned.py` falls back to the most recent `checkpoint-N/` subdirectory if the parent dir lacks a model file.
- **Bug 5 – Kaggle CLI silent download truncation.** On Windows, `kaggle kernels output` silently truncates filenames containing non-ASCII bytes and re-paginates listings such that duplicate downloads overwrite earlier good files with 0-byte placeholders. Fix: invoke the Python API with `PYTHONIOENCODING=utf-8` and paginate manually via `ApiListKernelSessionOutputRequest`.

After all five fixes, LED training ran cleanly through 11 epochs (Figure 1): evaluation loss falls monotonically while validation ROUGE-L rises from 0.107 to a peak of 0.172 at epoch 9, at which point early stopping fired at patience 2. The valid-only test ROUGE-L is 0.219 (Table 3). The clean, monotone curve is the clearest single piece of evidence that the earlier collapses were data and code defects rather than a modelling limit: nothing about the architecture or hyperparameters changed between the collapsing runs and this one — only the five bugs were removed.

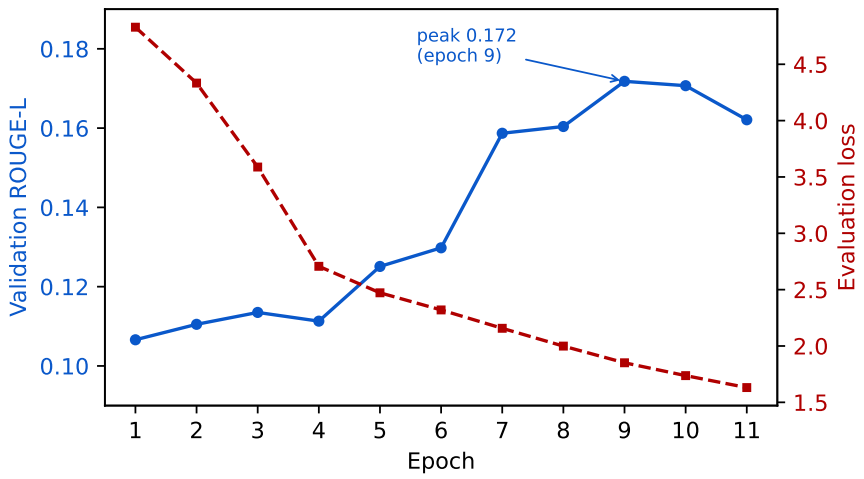

Figure 1: LED fine-tuning (run 3) after the five fixes of Section 8. Validation ROUGE-L (blue, left axis) rises to a peak of 0.172 at epoch 9; evaluation loss (red, right axis) decreases monotonically. Contrast with the pre-fix runs, in which ROUGE-L collapsed to zero by epoch 3 while loss kept falling.

8.2 Empty-reference dataset incident

The single most impactful fix in the project came from *inspecting the raw inputs*.

- Generated summaries on Day 2 were hallucinating obvious facts despite ROUGE-L $\sim$ 0.14. Opening `2001_facup_arsenal_liverpool.txt` revealed it was 0 bytes.
- Audit: 33% of training references (26/80), 30% of validation references (3/10), and **44% of test references** (4/9) were empty.
- ROUGE on an empty reference is exactly 0. The reported test ROUGE-L was therefore $\sum_{\text{valid}}/9$ instead of $\sum_{\text{valid}}/5$, deflating every condition by a factor of $\sim$1.8$\times$. More damagingly: 33% of training pairs were (transcript, ""), teaching the model to emit nothing on a third of

inputs — the actual root cause of LED's collapse-to-EOS that no hyperparameter change had fixed.

- Fix: pull the upstream dataset 1.0 commit, which has all 99 references populated; re-run.
- Result: predicted versus measured improvement after re-run (Table 6). The measured ratios (1.7–1.9× across conditions) closely bracket the predicted ∼1.8×, confirming the empty-target problem accounted for nearly all of the previous underperformance.

Condition	Before fix	After fix	Δ
finetuned_bart	0.1361	0.2476	+82%
led_long_zero	0.0825	0.1599	+94%
flan_chunk_zero	0.0413	0.0700	+69%

Table 6: ROUGE-L before and after the empty-reference fix.

8.3 Naive truncated-input BART fine-tuning

The first BART fine-tune trained on `(transcript[:1024], gold)` pairs and reached ROUGE-L = 0.127. Replacing the input with the concatenation of pre-trained per-chunk summaries — matching the inference-time pipeline — raised ROUGE-L to 0.136 on the broken dataset and to 0.248 after the dataset fix. The combined train/inference distribution match plus two-phase optimisation accounts for the entire gap between naive and final fine-tuned BART.

9 Limitations

Small test set. With $n = 9$ matches, differences within ±0.02 ROUGE-L are not statistically distinguishable. We did not run bootstrap confidence intervals; doing so is a clear next step.

Hallucination is unmeasured. The qualitative analysis (Section 7) makes clear that fine-tuned models invent scores, dates, stadiums, and scorers — exactly the facts a match-report user cares about. ROUGE does not penalise this. A faithfulness measure — BERTScore-precision against the *transcript* rather than the reference, or NLI-based entailment — would expose the gap.

GPT-4 references introduce evaluation bias. References in Section 3.3 were generated by search-enabled GPT-4 (ChatGPT web) from match identifiers, not written from the transcript. Web retrieval grounds their facts in published match reports — BBC sources were verified per match — so outright factual errors are less likely than with purely parametric generation — but the references still describe the match as the written press covered it, not as the radio commentary presented it, and they exhibit a stylistic homogeneity that surface metrics reward. They are also not exactly reproducible, since the underlying search results drift over time. Our reported scores therefore measure agreement with a GPT-4-distilled synthesis of web coverage, not agreement with a human gold standard paired to the input.

BERTScore not reported. Kaggle's transformers + bert_score combination raises `OverflowError` on inputs of our length. We did not have time to debug the dependency conflict; a local re-run with a pinned bert_score version is straightforward.

Whisper noise is the irreducible floor. Systematic player-name errors ("Bergkamp" → "Burgkamp") propagate into every model output and depress every surface-overlap metric. The pre-2000 audio is markedly noisier than recent recordings, contributing to per-match variance.

$N = 80$ **is the binding training constraint.** Both fine-tuned models likely overfit. The validation-to-test gap (BART 0.21 → 0.14 on the broken set; 0.31 → 0.25 on the clean set, where clean validation ROUGE-L peaks at 0.308) supports this. More data is the highest-leverage improvement.

10 Conclusions

We built an end-to-end pipeline for summarising long football radio commentaries on a custom 99-match dataset, comparing two input strategies (chunk-and-aggregate vs. long-context LED) and two learning paradigms (prompting vs. fine-tuning) across three model families (FLAN-T5, BART, LED). Three findings stand out:

1. Fine-tuning BART as the merge step of a chunk-aggregate pipeline gives a +55% relative ROUGE-L improvement over the strongest prompting baseline, and +69% over the same pre-trained model under the same pipeline.
2. Among prompting baselines, long-context (LED-16384) dominates chunk-aggregate when no fine-tuning is applied; but among fine-tuned models, chunk-aggregate BART beats long-context LED, suggesting that long-context fine-tuning is more sample-hungry than chunk-aggregate fine-tuning at our data scale.
3. Across all model families and prompt styles, zero-shot prompts outperform few-shot and chain-of-thought variants — consistent with prompt-budget pressure in the chunk-aggregate pipeline and with LED-base's non-instruction-tuned pre-training in the long-context pipeline.

The largest single improvement in the project came not from a model change but from *detecting and fixing a 33% empty-reference rate in the training data*, which had been silently teaching all models to emit empty strings. Equally instructive, the qualitative analysis shows that all fine-tuned models hallucinate central match facts (scores, scorers, dates) while ROUGE happily rewards them. Reliable football-match summarisation requires a faithfulness signal that surface-overlap metrics cannot provide; designing that signal is the natural next step beyond this work.

References

Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 2020.

Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, et al. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022.

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, 2020.

Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, 2004.

Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1906–1919, 2020.

OpenAI. GPT-4 technical report, 2023.

Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning*, pages 28492–28518, 2023.

Abigail See, Peter J. Liu, and Christopher D. Manning. Get to the point: Summarization with pointer-generator networks. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pages 1073–1083, 2017.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations*, 2020.